SYNOPSIS

ON

AI Health Report Translator

Submitted By:

Team No -33

Kritika Yadav - FB -2415000857

Kumari Bhavya Singh- FD - 2415000863

Aditya Mishra -FG -2415000857

Anubhavi Sinha – FF -2415000285

Submitted To:

Ms. Meghana M

Technical Trainer

CEA

Title of the Project:

AI Health Report Translator: An AI-Powered Web Application for Simplified Interpretation and Trend Tracking of Medical Laboratory Reports

Objective:

Medical and laboratory reports are written for healthcare professionals and typically contain technical test names, abbreviations, units and reference ranges. Patients who receive such reports often find it difficult to understand which values lie within the expected range, which do not, and what questions they should raise during a consultation. Searching for explanations on the internet can add to the confusion, and because reports are usually kept as scattered paper or PDF files, comparing results across different dates is difficult.

The main objective of this project is to design and develop a secure web application, the **AI Health Report Translator**, that helps users understand their medical laboratory reports in simple, non-technical language. The application is intended strictly as an informational and educational assistant. It does not diagnose diseases, recommend treatment, or replace the advice of a qualified healthcare professional.

The specific objectives of the project are:

- To allow users to upload laboratory reports in PDF or image format and extract test names, values, units and reference ranges from them.
- To classify each extracted value as Normal, High or Low by comparing it with the reference range printed on the report.
- To generate plain-language explanations of each test and a concise summary of the overall report using an AI (Large Language Model) service, while avoiding diagnostic statements.
- To generate a set of relevant questions that the user can discuss with a doctor.
- To store reports securely and present changes in health parameters over time through charts and graphs.
- To protect user data through authentication, access control and privacy-oriented design.

Scope:

The scope of the project covers the design, development and testing of a full-stack web application for interpreting laboratory reports for informational purposes.

Within the Scope

- User registration, login and session management using JSON Web Tokens (JWT).
- Upload of laboratory reports in PDF and common image formats (such as JPG and PNG).

- Text extraction from digital PDFs and Optical Character Recognition (OCR) for scanned or image-based reports.
- Extraction of test names, result values, units and reference ranges, followed by a user review step to correct extraction errors.
- Rule-based classification of numeric results as Normal, High or Low against the reference range given in the report.
- AI-generated simple explanations, report summary and suggested questions for the doctor, with a clear disclaimer on every result.
- Storage of report history and visualisation of parameter trends over time using charts.

Outside the Scope

- Diagnosis of any disease or medical condition, prediction of illness, or recommendation of medicines, dosages or treatments.
- Replacement of consultation with qualified healthcare professionals or emergency medical services.
- Interpretation of medical imaging (such as X-ray, MRI or CT scans), handwritten prescriptions or clinical notes.
- Integration with hospital information systems, laboratories or electronic health record (EHR) platforms.
- Formal clinical validation, regulatory approval or certification as a medical device.

Limitations

- Laboratory report layouts differ widely; extraction accuracy may vary for unusual formats, low-quality scans, rotated images or handwritten content.
- OCR can misread characters or numbers. For this reason, the extracted values are shown to the user for confirmation before analysis.
- Classification depends on the reference range printed in the report. The system does not independently apply age-, gender- or laboratory-specific ranges.
- The prototype focuses on numeric test results. Qualitative results (for example, "Positive" or "Negative") are displayed as reported and are not graded.
- AI language models can occasionally produce inaccurate or incomplete explanations. Output is therefore restricted by prompt design and validation, and users are always advised to consult a qualified doctor.
- Trend comparison requires consistent test names and units across reports; values in different units may not be directly comparable.
- The system depends on the availability of a third-party AI API and an internet connection.

Methodology:

The project will follow an iterative (Agile-inspired) development approach, in which the system is built and tested in short cycles covering requirement analysis, design, implementation, testing and review. The application is based on a three-tier architecture consisting of a React front end, a Node.js/Express back end exposing REST APIs, and a MongoDB database, supported by report-processing and AI-service modules.

Technologies Used

Layer	Technology	Purpose
Frontend	React.js, Vite, JavaScript, HTML, CSS	Responsive user interface, routing and state handling; Vite provides the development server and build tooling
UI Components and Charts	Lucide React icons, Recharts (or a similar charting library)	Icons and graphical display of health parameter trends
Backend	Node.js, Express.js, REST APIs	Application logic, API endpoints, report-processing pipeline and AI-service integration
Authentication	JSON Web Tokens (JWT), password hashing (e.g., bcrypt)	Secure login and protected routes
File Handling	Multer	Validated multipart file uploads (type and size restrictions)
Report Processing	PDF text-extraction library, OCR engine (e.g., Tesseract)	Reading text from digital PDFs and scanned or image-based reports
Database	MongoDB	Storage of user accounts, reports, extracted results, analyses and trend data
AI Service	LLM/AI API [final provider to be selected]	Plain-language explanation, summary and doctor-question generation

Working Methodology

1. **User authentication:** The user registers and logs in. The server issues a signed JWT, which is required to access protected API routes.
2. **Report upload:** The user uploads a PDF or image. Multer validates the file type and size before it is accepted.
3. **Text extraction:** For PDFs containing a text layer, text is extracted directly. For scanned PDFs and images, the pages are pre-processed and passed to the OCR engine.
4. **Structured data extraction:** The extracted text is parsed to identify test names, values, units and reference ranges. Pattern-based parsing is used as the primary method, and the AI service may be used to assist in structuring ambiguous text.

5. **User verification:** The extracted values are shown in an editable table so that the user can correct any misread value before analysis.
6. **Status classification:** The back end compares each numeric value with its reference range using deterministic program logic and labels it Normal, High or Low. This step is deliberately performed in code rather than by the AI model, so that the classification is consistent and verifiable.
7. **AI analysis:** The verified test data is sent to the AI service with a carefully designed prompt. The prompt instructs the model to use simple language, to base its explanation only on the supplied values and ranges, to avoid diagnosing or naming diseases as a conclusion, to avoid recommending treatment, and to return its response in a structured JSON format. The response is validated before it is saved.
8. **Summary and doctor questions:** The AI service produces a short overall summary and a list of suggested questions for the user to discuss with a healthcare professional.
9. **Storage:** The report metadata, extracted results, status labels and generated explanations are stored in MongoDB against the user's account.
10. **Trend tracking:** Results for the same parameter across multiple reports are grouped using normalised test names and plotted against report dates, with the reference range shown for comparison.

Security and Privacy Considerations

- Passwords are stored only as salted hashes; authentication uses JWTs with a limited validity period.
- All data access is restricted to the authenticated user, and every database query is filtered by user identity.
- Uploaded files are checked for permitted types and size limits, and API inputs are validated.
- Data is transmitted over HTTPS in deployment, and secrets such as API keys and database credentials are kept in environment variables, never in source code.
- Personal identifiers (such as patient name, address and contact details) are not required for analysis and are excluded from the content sent to the external AI service wherever possible.
- Users can delete their reports and their account. Only the minimum data required for the features is stored.
- The design is guided by common web-security practices such as the OWASP Top Ten and by the principles of data minimisation and user consent under applicable data-protection laws.

Testing

The system will be tested using unit tests for the parsing and classification logic, API testing of the REST endpoints, and end-to-end testing of the complete workflow. A test set of synthetic or anonymised sample reports in different layouts will be used to evaluate extraction behaviour. Testing will also check that AI responses follow the non-diagnostic format, and basic usability feedback will be collected from peers.

Proposed System:

The proposed system is a web-based assistant that converts a technical laboratory report into a clear, easy-to-read explanation. After logging in, the user uploads a report and the system extracts the relevant test details. The user confirms the extracted values, after which each result is labelled Normal, High or Low using the reference range given in the report. An AI service then explains each value in simple language, writes a short summary of the report and suggests questions to ask a doctor. The results are saved so that the user can later view report history and observe how parameters change over time.

Core Workflow

User Registration/Login → Dashboard → Upload Medical Report → PDF/Image Processing → Extraction of Test Names, Values, Units and Reference Ranges → AI Analysis → Identification of Normal/High/Low Values → Simple Summary → Questions for Doctor → Report Storage → Health Trend Tracking

System Modules

Module	Description
Authentication Module	Handles registration, login, JWT issuing and verification, and route protection.
Report Upload Module	Accepts and validates report files using Multer and links them to the user account.
Report Processing Module	Performs PDF text extraction or OCR and converts the text into structured test records.
Analysis Module	Classifies values as Normal, High or Low, and calls the AI service for explanations, summary and doctor questions.
Trend and Visualisation Module	Groups results by parameter across reports and presents them as charts.
Data Management Module	Maintains user, report and result data in MongoDB, including report history and deletion.

Illustrative Example

If a report contains **WBC Count = 11,200** with a **Reference Range of 4,000–11,000**, the system marks the value as **High** because it is above the provided range. The explanation will be worded

cautiously, for example: "Your WBC count is slightly above the provided reference range. There can be several reasons for an elevated WBC count. Discuss this result with a qualified healthcare professional." The system will not make statements such as "You have an infection", since it does not diagnose conditions.

Every analysis screen will display a disclaimer stating that the application is an informational and educational tool and not a substitute for professional medical advice, diagnosis or treatment.

Features:

- **User Authentication:** Secure registration and login using JWT-based authentication.
- **Medical Report Upload:** Upload of laboratory reports in PDF or image format with file validation.
- **PDF/Image Report Processing:** Text extraction from PDFs and OCR for scanned or image-based reports.
- **AI-Based Report Analysis:** Interpretation of extracted test data using an LLM/AI API.
- **Simple Language Explanation:** Plain-language description of each test and what the result generally indicates.
- **Abnormal Value Detection:** Labelling of each value as Normal, High or Low based on the reference range in the report, with abnormal values highlighted.
- **Health Summary:** A short, easy-to-read overview of the whole report.
- **Doctor Q&A / Suggested Questions:** A list of relevant questions the user can take to a medical consultation.
- **Health Trend Tracking:** Comparison of the same parameter across multiple reports over time.
- **Graphical Visualisation:** Line charts of parameter values with reference range markers.
- **Report History:** A dashboard listing previously uploaded reports, which can be viewed or deleted.
- **Secure and Private User Data:** Access control, hashed passwords, input validation, minimal data sharing and user-controlled deletion.
- **Result Verification and Disclaimer:** Editable extracted values before analysis and a visible non-diagnostic disclaimer on every result.

Implementation Plan:

The project is planned over an estimated period of 16 weeks, divided into the phases and milestones below. The timeline can be adjusted to match the academic calendar of the university.

Phase	Duration	Activities	Milestone
1	Weeks 1–2	Problem definition, requirement analysis, literature survey, finalising the technology stack and AI service	Requirement document
2	Weeks 3–4	System design: architecture, database schema, REST API design, UI wireframes, prompt design guidelines	Design document
3	Weeks 5–6	Project setup, user authentication, dashboard layout, report upload with Multer	Login and upload working
4	Weeks 7–9	PDF text extraction, OCR integration, parsing of tests, values, units and ranges, verification screen, Normal/High/Low classification	Extraction pipeline working
5	Weeks 10–11	AI API integration, prompt refinement, simple explanations, summary and doctor-question generation, response validation	AI analysis working
6	Weeks 12–13	Report storage and history, test-name normalisation, trend charts using Recharts	Trend tracking working
7	Weeks 14–15	Security hardening, unit/API/integration testing, bug fixing, UI refinement	Tested prototype
8	Week 16	Deployment, user documentation, final report and presentation	Final submission

Team Members:

S.No.	Name	University Roll No.	Role
1	Kritika Yadav	2415000857	Team Leader; Backend and API development
2	Anubhavi Sinha	2415000285	Frontend development and data visualisation
3	Kumari Bhavya Singh	2415000863	Report processing (PDF/OCR) and AI integration
4	Aditya Mishra	2415000103	Database design, testing and documentation

Resources Required:

Hardware

- Development computers with at least 8 GB RAM and a modern multi-core processor.
- Stable internet connection for AI API access and package installation.

Software

- Operating system: Windows, Linux or macOS.
- Node.js (LTS version), npm, and Visual Studio Code (or a similar editor).
- React.js, Vite, Express.js, Multer, JWT and password-hashing libraries, Recharts, Lucide React.
- MongoDB (local installation or a cloud-hosted instance) and a database viewer.
- PDF text-extraction library and an OCR engine such as Tesseract.
- Git and GitHub for version control; Postman (or similar) for API testing; a modern web browser.

External Services and Data

- An LLM/AI API account and API key [provider to be finalised].
- A cloud hosting platform for demonstration deployment [to be selected].
- Synthetic or anonymised sample laboratory reports for development and testing, obtained with proper consent. No real patient data will be used without permission.

Human Resources

- Team members as listed above, under the guidance of the project supervisor.

References:

[1] Meta Platforms, Inc., "React Documentation," [Online]. Available: https://react.dev

[2] Vite Team, "Vite: Getting Started," [Online]. Available: https://vite.dev/guide/

[3] OpenJS Foundation, "Node.js Documentation," [Online]. Available: https://nodejs.org/docs/latest/api/

[4] OpenJS Foundation, "Express.js Documentation," [Online]. Available: https://expressjs.com/

[5] MongoDB, Inc., "MongoDB Manual," [Online]. Available: https://www.mongodb.com/docs/manual/

[6] Express.js Team, "Multer: Node.js middleware for handling multipart/form-data," GitHub. [Online]. Available: https://github.com/expressjs/multer

[7] Recharts, "Recharts Documentation," [Online]. Available: https://recharts.org/

[8] Lucide Contributors, "Lucide React Guide," [Online]. Available: https://lucide.dev/guide/packages/lucide-react

[9] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), May 2015.

[10] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in Proc. USENIX Annual Technical Conference, 1999.

[11] R. Smith, "An Overview of the Tesseract OCR Engine," in Proc. 9th Int. Conf. on Document Analysis and Recognition (ICDAR), 2007, pp. 629–633.

[12] Mozilla, "PDF.js Documentation," [Online]. Available: https://mozilla.github.io/pdf.js/

[13] Anthropic, "Claude API Documentation," [Online]. Available: https://docs.claude.com/ (or the documentation of the AI provider finally selected).

[14] A. Vaswani et al., "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[15] Z. Ji et al., "Survey of Hallucination in Natural Language Generation," ACM Computing Surveys, vol. 55, no. 12, Art. 248, 2023.

[16] K. Singhal et al., "Large language models encode clinical knowledge," Nature, vol. 620, pp. 172–180, 2023.

[17] A. Rajkomar, J. Dean and I. Kohane, "Machine Learning in Medicine," New England Journal of Medicine, vol. 380, no. 14, pp. 1347–1358, 2019.

[18] E. J. Topol, "High-performance medicine: the convergence of human and artificial intelligence," Nature Medicine, vol. 25, pp. 44–56, 2019.

[19] OWASP Foundation, "OWASP Top Ten," [Online]. Available: https://owasp.org/www-project-top-ten/

[20] Government of India, "The Digital Personal Data Protection Act, 2023," Ministry of Electronics and Information Technology.

Expected Outcomes:

By the end of the project, the team expects to deliver:

- A working web application that accepts laboratory reports in PDF or image format and extracts test names, values, units and reference ranges.
- Clear labelling of results as Normal, High or Low, with easy-to-understand explanations and a report summary that do not make diagnostic claims.
- Suggested questions that help users prepare for discussions with their doctor.
- A secure report history and chart-based trend tracking for health parameters over time.
- A privacy-conscious system design with authentication, access control and user-controlled data deletion.
- Project documentation, including the final report, design documents and test results.

The project is expected to improve the accessibility of health information for non-expert users and to give the team practical experience in full-stack development, document processing and responsible use of AI in a sensitive domain.

Project Supervisor:

Name: Meghna M
Designation: Technical Trainer
Department: CEA
Institution: GLA University, Mathura

Conclusion:

Laboratory reports contain valuable information, but their technical presentation often prevents patients from understanding them. The AI Health Report Translator aims to bridge this gap by combining report processing, rule-based value classification and AI-generated plain-language explanations in a secure web application. By storing past reports and visualising changes over time, it also helps users follow their health parameters more easily.

The system is designed as an informational and educational tool only. It does not diagnose diseases or replace professional medical advice; instead, it encourages users to discuss their results with qualified healthcare professionals by preparing them with a better understanding and relevant questions. With a practical technology stack (React, Node.js, Express, MongoDB, OCR and an AI API) and a phased implementation plan, the project is realistic for completion within the academic term.